

Обход в ширину

Обход в ширину (Поиск в ширину, англ. *BFS*, *Breadth-first search*) — один из простейших алгоритмов обхода графа, являющийся основой для многих важных алгоритмов для работы с графами.

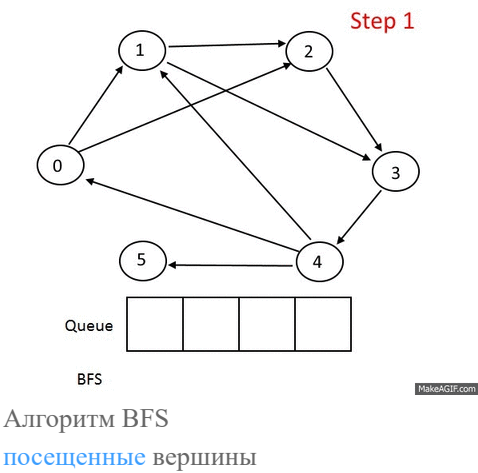
Содержание

- 1 Описание алгоритма
- 2 Анализ времени работы
- 3 Корректность
- 4 Дерево обхода в ширину
- 5 Реализация
- 6 Вариации алгоритма
 - 6.1 0-1 BFS
 - 6.2 1-k BFS
- 7 См. также
- 8 Источники информации

Описание алгоритма

Пусть задан невзвешенный ориентированный граф $G = (V, E)$, в котором выделена исходная вершина s . Требуется найти длину кратчайшего пути (если таковой имеется) от одной заданной вершины до другой. Частным случаем указанного графа является невзвешенный неориентированный граф, т.е. граф, в котором для каждого ребра найдется обратное, соединяющее те же вершины в другом направлении.

Для алгоритма нам потребуются очередь и множество посещенных вершин was , которые изначально содержат одну вершину s . На каждом шагу алгоритм берет из начала очереди вершину v и добавляет все непосещенные смежные с v вершины в was и в конец очереди. Если очередь пуста, то алгоритм завершает работу.



Анализ времени работы

Оценим время работы для входного графа $G = (V, E)$, где множество ребер E представлено списком смежности. В очередь добавляются только непосещенные вершины, поэтому каждая вершина посещается не более одного раза. Операции внесения в очередь и удаления из нее требуют $O(1)$ времени, так что общее время работы с очередью составляет $O(|V|)$ операций. Для каждой вершины v рассматривается не более $\deg(v)$ ребер, инцидентных ей. Так как $\sum_{v \in V} \deg(v) = 2|E|$, то время, используемое на работу с ребрами, составляет $O(|E|)$. Поэтому общее время работы алгоритма поиска в ширину — $O(|V| + |E|)$.

Корректность

Утверждение:

В очереди поиска в ширину расстояние вершин до s монотонно неубывает.

▷

Докажем это утверждение индукцией по числу выполненных алгоритмом шагов.

Введем дополнительный инвариант: у любых двух вершин из очереди, расстояние до s отличается не более чем на 1.

База: изначально очередь содержит только одну вершину s .

Переход: пусть после i — й итерации в очереди $a + 1$ вершин с расстоянием x и b вершин с расстоянием $x + 1$.

Рассмотрим i — ю итерацию. Из очереди достаём вершину v , с расстоянием x . Пусть у v есть r непосещённых смежных вершин. Тогда, после их добавления, в очереди находится a вершин с расстоянием x и, после них, $b + r$ вершин с расстоянием $x + 1$.

Оба инварианта сохранились, $\Rightarrow$ после любого шага алгоритма элементы в очереди неубывают.
◁

Теорема:

Алгоритм поиска в ширину в невзвешенном графе находит длины кратчайших путей до всех достижимых вершин.

Доказательство:

▷

Допустим, что это не так. Выберем из вершин, для которых кратчайшие пути от s найдены некорректно, ту, настоящее расстояние до которой минимально. Пусть это вершина u , и она имеет своим предком в дереве обхода в ширину v , а предок в кратчайшем пути до u — вершина w .

Так как w — предок u в кратчайшем пути, то $\rho(s, u) = \rho(s, w) + 1 > \rho(s, w)$, и расстояние до w найдено верно, $\rho(s, w) = d[w]$. Значит, $\rho(s, u) = d[w] + 1$.

Так как v — предок u в дереве обхода в ширину, то $d[u] = d[v] + 1$.

Расстояние до u найдено некорректно, поэтому $\rho(s, u) < d[u]$. Подставляя сюда два последних равенства, получаем $d[w] + 1 < d[v] + 1$, то есть, $d[w] < d[v]$. Из ранее доказанной леммы следует, что в этом случае вершина w попала в очередь и была обработана раньше, чем v . Но она соединена с u , значит, v не может быть предком u в дереве обхода в ширину, мы пришли к противоречию, следовательно, найденные расстояния до всех вершин являются кратчайшими.

◁

Дерево обхода в ширину

Поиск в ширину также может построить дерево поиска в ширину. Изначально оно состоит из одного корня s . Когда мы добавляем непосещённую вершину в очередь, то добавляем её и ребро, по которому мы до неё дошли, в дерево. Поскольку каждая вершина может быть посещена не более одного раза, она имеет не более одного родителя. После окончания работы алгоритма для каждой достижимой из s вершины t путь в дереве поиска в ширину соответствует кратчайшему пути от s до t в G .

Реализация

Предложенная ниже функция возвращает кратчайшее расстояние между двумя вершинами.

- **source** — исходная вершина
- **destination** — конечная вершина
- **G** — граф, состоящий из списка вершин **V** и списка смежности **E**. Вершины нумеруются целыми числами.
- **Q** — очередь.
- В поле **d[u]** хранится расстояние от **source** до **u**.

```
int BFS(G: (V, E), source: int, destination: int):
    d = int[|V|]
    fill(d, ∞)
    d[source] = 0
    Q = ∅
    Q.push(source)
    while Q ≠ ∅
        u = Q.pop()
        for v: (u, v) in E
            if d[v] == ∞
                d[v] = d[u] + 1
                Q.push(v)
    return d[destination]
```

Если требуется найти расстояние лишь между двумя вершинами, из функции можно выйти, как только будет установлено значение **d[destination]**. Еще одна оптимизация может быть проведена при помощи метода meet-in-the-middle.

Вариации алгоритма

0-1 BFS

Пусть в графе разрешены ребра веса **0** и **1**, необходимо найти кратчайший путь между двумя вершинами. Для решения данной задачи модифицируем приведенный выше алгоритм следующим образом:

Вместо очереди будем использовать дек (или можно даже steque). Если рассматриваемое ее ребро имеет вес **0**, то будем добавлять вершину в начало, а иначе в конец. После этого добавления, дополнительный введенный инвариант в доказательстве расположения элементов в деке в порядке неубывания продолжает выполняться, поэтому порядок в деке сохраняется. И, соответственно, релаксируем расстояние до всех смежных вершин и, при успешной релаксации, добавляем их в дек.

Таким образом, в начале дека всегда будет вершина, расстояние до которой меньше либо равно расстоянию до остальных вершин дека, и инвариант расположения элементов в деке в порядке неубывания сохраняется. Значит, алгоритм корректен на том же основании, что и обычный BFS. Очевидно, что каждая вершина войдет в дек не более двух раз, значит, асимптотика у данного алгоритма та же, что и у обычного BFS.

1-k BFS

Пусть в графе разрешены ребра целочисленного веса из отрезка **1 . . . k**, необходимо найти кратчайший путь между двумя вершинами. Представим ребро *uv* веса *m* как последовательность ребер *uu₁ u₂ . . . u_{m−1} v* (где *u₁ . . . u_{m−1}* — новые вершины). Применим данную операцию ко всем ребрам графа *G*. Получим граф, состоящий (в худшем случае) из *k|E|* ребер и *|V| + (k − 1)|E|* вершин. Для нахождения кратчайшего пути следует запустить BFS на новом графе. Данный алгоритм будет иметь асимптотику *O(|V| + k|E|)*.

См. также

- Обход в глубину, цвета вершин
- Алгоритм Дейкстры
- Теория графов

Источники информации

- Томас Х. Кормен, Чарльз И. Лейзерсон, Рональд Л. Ривест, Клиффорд Штайн Алгоритмы: построение и анализ — 2-е изд. — М.: «Вильямс», 2007. — с. 459. — ISBN 5-8489-0857-4
- MAXimal :: algo :: Поиск в ширину (<http://e-maxx.ru/algo/bfs>)
- Wikipedia — Breadth-first search
- Wikipedia — Поиск в ширину
- Визуализатор алгоритма (<http://rain.ifmo.ru/cat/view.php/vis/graph-general/bfs-2002>)

Источник — «http://neerc.ifmo.ru/wiki/index.php?title=Обход_в_ширину&oldid=85552»

-
- Эта страница последний раз была отредактирована 4 сентября 2022 в 19:35.